Application Packaging & Deployment Standards (Win32, MSIX & Store) — Org-Wide Specification & Quality Gates

Implementation Solution Guide

Version 1.0.0 · 2026-07-20

Contents

Executive Summary — Application Packaging & Deployment Standards (Win32, MSIX & Store)

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Application Packaging & Deployment Standards (Win32, MSIX & Store) 5

What this solution is . . . 5

The problem it solves . . . 5

The four control points . . . 6

The three app formats you will govern . . . 6

What the standard actually produces . . . 6

Benefits . . . 7

Limitations (be honest) . . . 7

The standard deployment model . . . 8

2. Requirements Guide — Application Packaging & Deployment Standards 8

2.1 Intune licensing for every targeted user or device . . . 8

2.2 A named owner for the packaging standard . . . 8

2.3 At least two people who can package and review . . . 9

2.4 Microsoft Graph PowerShell SDK on the auditor's workstation . . . 9

2.5 Graph permissions for the audit account . . . 9

2.6 Intune RBAC roles separating packagers from approvers . . . 10

2.7 A document store for retained package evidence . . . 10

2.8 A defined pilot population that is real . . . 10

2.9 A confirmed position on Microsoft Store (new) availability in your cloud . . . 10

2.10 An agreed application catalog taxonomy . . . 11

2.11 A record of which existing apps are in scope for retrofit . . . 11

Checklist . . . 12

Global Requirements — Intune Solutions (category-level) 12

G1. Microsoft Entra ID tenant . . . 12

G2. Microsoft Intune subscription . . . 12

G3. Administrative roles . . . 12

G4. Device enrollment in Intune . . . 12

G5. Pilot security group . . . 13

G6. Supported Windows baseline . . . 13

G7. Network access to Microsoft cloud endpoints . . . 13
3. Implementation Guide — Application Packaging & Deployment Standards 13
Named objects (reuse these names verbatim everywhere) . . . 13
Step 1 — Create the evidence store . . . 14
Step 2 — Create the tenant objects the standard depends on . . . 15
Step 3 — The format decision matrix (THE decision tree) . . . 16
Step 4 — The packaging specification (naming, versioning, metadata, documentation) . . . 18
Step 5 — Detection-rule standards . . . 20
Step 6 — Install / uninstall command, return-code, and reboot standards . . . 22
Step 7 — The Package Intake & Specification Template . . . 24
Step 8 — The package quality gates (acceptance checklist) . . . 25
Step 9 — Peer review, sign-off, and the release process . . . 27
Step 10 — Enforcement and audit over time . . . 28
5. Testing Guide — Application Packaging & Deployment Standards 30
Part A — The standard is in place . . . 30
Part B — A package passes through the standard . . . 31
Pass/fail gate . . . 32
6. Troubleshooting Guide — Application Packaging & Deployment Standards 32
A. The scripts won't connect or return nothing . . . 32
B. An app fails the naming/standard audit but "looks fine" . . . 33
C. Detection quality checker flags a rule . . . 33
D. Assignment hygiene report shows violations . . . 34
E. Metadata remediation didn't change anything . . . 34
F. The standard is "adopted" but nobody follows it . . . 35
G. Government-cloud specifics . . . 35
H. When to escalate . . . 35
7. Best Practices — Application Packaging & Deployment Standards 35
8. Deliverables Summary — Application Packaging & Deployment Standards 37
Documentation . . . 37
PowerShell scripts (Microsoft Graph, read-only by default) . . . 38
Reusable templates (inside the Implementation Guide) . . . 38
What the buyer receives . . . 38
Outcome . . . 39
Appendix A — PowerShell Scripts 40

Executive Summary — Application Packaging & Deployment Standards (Win32, MSIX & Store)

Intune Solutions

An application catalog decays predictably. The first ten apps are fine because one person built them all. By app forty there are three naming styles, detection rules only one person understands, apps assigned to groups nobody can explain, several apps with no owner, and a `Test - do not use` app that has been pushed to every device for months. Nobody did anything wrong — there was simply no **standard**. The cost is real: failed installs nobody can triage, licence spend on software still deployed to people who left, audit findings because there is no record of what was approved, and a "don't touch that app" culture where the safest move is to build a duplicate.

This solution is the **organisation-wide packaging specification** that prevents that decay. It sits **above** the mechanics of packaging any single application — it does not re-teach how to wrap a `.intunewin`; it governs how *every* app is named, formatted, detected, documented, reviewed, released, and audited across the whole tenant, for years.

It is written for an administrator with no prior standards experience, and it is deploy-it-this-week.

What you get

- A **Solution Overview** of the standards layer, its four control points (intake, build, release, operate), and the three delivery formats you govern (Win32, MSIX, Microsoft Store) plus the legacy LOB/MSI type being retired.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite, including the Government-cloud position on Microsoft Store availability.
- An **Implementation Guide** that stands up the tenant objects, then hands you the specification itself: a **format decision tree**, the **naming / version-stamping / metadata** standard, the **detection-rule acceptability rules and ten banned patterns**, the **command / return-code / reboot** standards, a reusable **Package Intake & Specification template**, a five-gate **acceptance checklist**, a **peer-review and sign-off** process, **assignment-hygiene** rules, and an **enforcement and audit** cadence.
- **Five PowerShell scripts** — a standard-compliance auditor, a detection-rule quality checker, an assignment-hygiene reporter, a metadata completeness scorer, and a preview-by-default metadata remediation tool — all using the Microsoft Graph PowerShell SDK, read-only except the remediation tool, which changes nothing without `-Apply`.
- **Testing** and **Troubleshooting** guides with explicit pass/fail gates.

What this delivers

A packaging standard that makes the catalog consistent, auditable, and clean at scale: every app correctly named and version-stamped in three agreeing places, every detection rule provably correct and free of the patterns that silently corrupt reporting, every release peer-reviewed with retained evidence, every assignment hygienic, and the whole catalog audited on a cadence by the included scripts.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand.** No package skips the acceptance checklist, and no package reaches broad release without a signed sign-off record.

Outcome

An app catalog that a new starter can pick up in days, an auditor can answer questions about in minutes, and a growing team can extend without it turning into a liability — because the standard, the templates, the quality gates, and the audit scripts are all in place from the first package onward.

1. Solution Overview — Application Packaging & Deployment Standards (Win32, MSIX & Store)

What this solution is

This is the **organisation-wide specification** that governs how every application in your tenant is packaged, named, detected, documented, reviewed, released, and audited. It is the layer **above** the mechanics of packaging any single app.

It does **not** teach you how to run `IntuneWinAppUtil.exe`, how to type an `msiexec` command, or how to click through the Win32 app wizard for one application — that is per-app packaging work. This solution answers the questions that only appear when **many people package many apps over many years**:

- What must an app be **named**, and why does an inconsistent catalog become unmanageable?
- Which **format** — Win32, MSIX, Microsoft Store (new), or LOB/MSI — is correct for a given app, and who decides?
- What makes a detection rule **acceptable and auditable**, and which patterns are **banned outright**?
- What must be true before a package is allowed to be **released** to real users?
- Who **reviews and signs off** a package, and what evidence is retained?
- How do assignments stay clean as the catalog grows from 10 apps to 300?
- How is the standard **enforced and audited** so it does not decay into a document nobody follows?

The problem it solves

An app catalog decays predictably. The first ten apps are fine because one person built them all. By app forty there are three naming styles, two people who "know" the detection rules, apps assigned to groups nobody can explain, four apps with no owner, and a `Test - do not use` app that has been assigned to All Devices for eight months. Nobody did anything wrong; there was simply no **standard**.

The cost is real and measurable: failed installs nobody can triage, licence spend on apps still deployed to leavers, audit findings because there is no record of what was approved, and a "do not touch that app" culture where the safest action is to create a duplicate.

A packaging standard is cheap to adopt and expensive to retrofit. This solution gives you the standard, the templates that operationalise it, the quality gates that enforce it, and the scripts that audit it.

The four control points

Everything in this specification hangs off four control points. Each is a place where a decision is made and where a rule can be enforced.

Control point	The question	Enforced by
1. Intake	Should we package this app at all, and in which format?	The Package Intake & Specification Template + the format decision matrix
2. Build	Does the package meet the naming, metadata, detection, command, and return-code standards?	The packaging specification + the acceptance checklist
3. Release	Has a second person reviewed it, and is the evidence retained?	The peer review & sign-off process
4. Operate	Are assignments still clean, and does the catalog still comply?	The ring/assignment standard + the quarterly audit scripts

The three app formats you will govern

The specification covers three delivery formats plus one legacy format being retired. The **format decision matrix** in the Implementation Guide (Step 3) is the authoritative decision tree; this is the orientation.

• **Win32** (`.intunewin`, installed by the Intune Management Extension) — the default. Wraps any

MSI, EXE, or script. Full control over install/uninstall commands, detection, requirements, dependencies, supersedence, and return codes. Chosen unless another format is clearly better.

• **MSIX** (and `.msixbundle` / `.appx`) — a modern, containerised Windows package format installed

by the OS rather than by a custom command line. No install command to get wrong, clean uninstall, and per-user by design. Only usable when the vendor ships an MSIX or when your packaging team converts one — and conversion is a project, not a step.

• **Microsoft Store (new)** — Intune's integration with the Microsoft Store, covering both UWP and,

since the "new" Store, packaged Win32 apps. Zero packaging effort and automatic vendor updates, which is exactly why it should be preferred where it is available and where automatic updating is acceptable to the business. **Availability differs in US Government clouds — see Requirement 2.9.**

• **Line-of-business (LOB) / MSI** — the legacy `.msi` app type. It exists, it still works, and the

standard's position is that it is **not used for new packages**. It is listed in the matrix only so that reviewers know to reject it and so that migration of existing LOB apps is a tracked item.

What the standard actually produces

When this specification is adopted, the following exist in your tenant and your document store, and they are the deliverables a reviewer or auditor will ask for:

1. A **naming convention** applied to every app, every assignment group, and every scope tag — `<Vendor> <Product> – <Format> – v<Version>` for apps and `APP – <Purpose> – <Stage>` for groups.
1. A completed **Package Intake & Specification form** per application, retained for its lifetime.
2. A **detection rule** per app that satisfies the acceptability rules and matches no banned pattern.
3. A signed **Package Acceptance Checklist** per release, with the reviewer's name and date.
4. A **ring assignment** — Ring 0 packaging lab, Ring 1 pilot, Ring 2 broad — with an explicit, named exclusion group.
1. A **quarterly compliance report** produced by the audit scripts, with the exceptions register.

Benefits

• **Anyone can pick up anyone else's package.** Consistent names, consistent metadata, and a completed spec form mean the person who packaged an app is not a single point of failure.

• **Detection stops being the top failure cause.** A rule that must pass documented acceptability tests is a rule that reports the truth.

• **Bad packages cannot reach production.** The acceptance checklist and peer sign-off are gates, not suggestions, and Ring 1 is a real ring with real users.

• **The catalog stays clean at scale.** Assignment hygiene rules plus a scripted quarterly audit stop orphan apps, duplicate assignments, and conflicting Required/Uninstall targeting accumulating.

• **Audit answers itself.** "Who approved this, when, against what test evidence?" has a filed answer for every app.

• **New starters are productive in days.** The standard plus the intake template is the onboarding material.

Limitations (be honest)

• **A standard is a social artefact.** Scripts can detect violations; only management support makes people fix them. The enforcement section (Implementation Step 10) is not optional garnish.

• **Retrofitting an existing catalog takes effort.** The remediation script normalises metadata, but renaming apps and rebuilding non-compliant detection rules is manual work. Plan it as a project and use the exceptions register for what you will not fix.

• **Microsoft Graph reflects what Intune stores, not what a human meant.** The audit scripts detect structural violations (missing publisher, banned detection pattern, orphan assignment); they cannot judge whether a description is *good*.

- **Format availability changes.** The Microsoft Store (new) app type in particular has different availability across Commercial and US Government clouds — the matrix tells you to verify before committing, and Requirement 2.9 covers the government position.
- **This standard governs deployment, not licensing.** Entitlement, licence counting, and app contract management are adjacent disciplines and out of scope.

The standard deployment model

> ***Intake the app against the spec form → decide the format from the matrix → build to the packaging specification → pass the acceptance checklist → peer review and sign-off → release to Ring 1 pilot → validate → expand to Ring 2 broad → audit quarterly.*** *This is the repeatable way:* ***Assess → Configure → Pilot → Validate → Expand.*** *No package skips the checklist, and no package reaches Ring 2 without a signed sign-off record.*

2. Requirements Guide — Application Packaging & Deployment Standards

Verify **every** requirement before you publish the standard to your packaging team. Each entry: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

Throughout this solution we use a **worked example application** so every rule is concrete:

> `Acme Invoice Client`*, version* ***7.3.1****, from publisher* `Acme Software Ltd.`*, shipped as an MSI (*`AcmeInvoiceClient.msi`*), business-owned by the Finance department, requested for the Finance and Accounts Payable teams.*

Replace the vendor, product, version, and owner with your own. The **names** of the tenant objects you create (groups, scope tags, categories, roles) are part of the standard — use them **verbatim** so that the scripts, the templates, and the audit all agree.

2.1 Intune licensing for every targeted user or device

Description: An Intune-bearing licence plan (Intune Plan 1, Microsoft 365 E3/E5, or Business Premium) assigned to every user or device that will receive a managed application. **Why required:** App deployment, app assignment, and the Intune Management Extension are Intune capabilities. Unlicensed users silently fail to receive Required assignments, which looks like a packaging fault and wastes triage time. **How to verify:** Microsoft 365 admin center (`https://admin.microsoft.com`) → **Billing** → **Licenses** → confirm an Intune-bearing SKU with available seats. Then Microsoft Intune admin center (`https://intune.microsoft.com`) → **Tenant administration** → **Tenant status** → confirm the tenant is active and note the **Account status**. **How to implement if missing:** Purchase and assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1 to the target users, then re-check **Tenant status**.

2.2 A named owner for the packaging standard

Description: One accountable person (with a named deputy) who owns this specification: approves changes to it, arbitrates format decisions, and chairs the exceptions register. **Why required:** A standard without an owner is a document. Every rule below has edge cases; someone must be able to say "yes, this exception is approved, here is why, review it in six months." Without this role the standard decays within two quarters. **How to verify:** The owner and deputy are named on the front page of your published copy of this specification, and both appear as **Owners** of the Entra group `ROLE – Package Approvers` (created in Implementation Step 2). **How to implement if missing:** Nominate the senior endpoint engineer as **Packaging Standard Owner** and one peer as deputy. Record both names in your published copy of the standard and add them as Owners of `ROLE – Package Approvers`.

2.3 At least two people who can package and review

Description: A minimum of two staff competent to build a package **and** to review someone else's. **Why required:** The peer review gate (Implementation Step 8) requires that the reviewer is **not** the packager. With one person, sign-off is self-certification and the gate is theatre. Two is the minimum; three removes the single point of failure when one is on leave. **How to verify:** Entra admin center (`https://entra.microsoft.com`) → **Groups** → **All groups** → open `ROLE – Package Approvers` → **Members** → confirm at least two members. **How to implement if missing:** Train a second administrator on the per-app packaging mechanics (that is a separate discipline — see the companion Win32 packaging solution) and add them to `ROLE – Package Approvers`. Until you have two, record it as an open risk in the exceptions register and have a manager counter-sign releases.

2.4 Microsoft Graph PowerShell SDK on the auditor's workstation

Description: The Microsoft Graph PowerShell SDK installed on the workstation that runs the five audit and remediation scripts in this package. **Why required:** Every script in this solution reads (and, for the remediation script, writes) app objects, assignments, and detection rules through Microsoft Graph. There is no portal report that answers "which apps violate the naming standard". **How to verify:** Open PowerShell and run:

```
Get-Module -ListAvailable -Name Microsoft.Graph.Authentication
```

A version is returned. Then run `.\APPSTD-Invoke-PackageStandardAudit.ps1 -WhatIfOnlySummary` and confirm it connects without error. **How to implement if missing:**

```
Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber
```

Use PowerShell 7.x where possible. All five scripts install the SDK automatically if it is absent, but pre-installing avoids a long first run.

2.5 Graph permissions for the audit account

Description: The account running the scripts must consent to `DeviceManagementApps.Read.All` (all five scripts) and additionally `DeviceManagementApps.ReadWrite.All` for the metadata remediation script, plus `Group.Read.All` for the assignment hygiene report. **Why required:** Read scopes let the auditor enumerate apps, assignments, and detection rules. The write scope is needed **only** by `APPSTD-Set-AppMetadataStandard.ps1`, which is `-WhatIf` by default and changes nothing until you explicitly pass `-Apply`. **How to verify:** Run any audit script; on first run PowerShell opens a consent prompt listing the scopes. If consent has already been granted, run:

```
Connect-MgGraph -Scopes 'DeviceManagementApps.Read.All','Group.Read.All' -NoWelcome
```

```
(Get-MgContext).Scopes
```

and confirm the scopes are listed. **How to implement if missing:** Sign in as a user holding the **Intune Administrator** role and accept the consent prompt. If your tenant blocks user consent, have a Global Administrator grant admin consent for the Microsoft Graph PowerShell enterprise application for those scopes.

2.6 Intune RBAC roles separating packagers from approvers

Description: Distinct Intune role assignments so that the people who build packages and the people who approve releases are visibly different sets, and neither group has more rights than the job needs. **Why required:** The sign-off gate has integrity only if it is enforced by permissions as well as by process. It also limits blast radius: a packager should not be able to assign an app to All Devices. **How to verify:** Intune admin center → **Tenant administration** → **Roles** → **All roles** → confirm a custom role named `Application Packager (Standard)` exists, and open **Assignments** to confirm it is assigned to your packagers and scoped by the scope tag `SCOPE – Application Packaging`. **How to implement if missing:** Create the role and scope tag in Implementation Step 2. If your organisation is too small to separate the duties, document that as an approved exception in the exceptions register and rely on the peer-review signature instead — but say so explicitly rather than leaving the gap undeclared.

2.7 A document store for retained package evidence

Description: A durable, access-controlled location (a SharePoint document library, a Teams files tab, or a version-controlled repository) that holds one folder per application containing the intake form, the acceptance checklist, the sign-off record, and the test evidence. **Why required:** The standard's audit answer — "who approved this, when, against what evidence" — depends on the evidence still existing in two years. Evidence in a packager's `C:\Temp` is not evidence. **How to verify:** The library exists, is named `Application Package Records`, is readable by `ROLE – Package Approvers` and your audit function, and contains at least one folder using the convention `<Vendor> <Product>\<Version>\` (for the worked example, `Acme Software Ltd Acme Invoice Client\7.3.1\`). **How to implement if missing:** Create a SharePoint document library named `Application Package Records`. Turn on **version history**. Grant **Contribute** to your packaging team and **Read** to audit/compliance. Do not allow deletion by contributors.

2.8 A defined pilot population that is real

Description: A pilot ring of genuine, representative end users — not only the IT team — who receive every package before broad release. Aim for 3–5% of the fleet, minimum 10 devices, spanning your main hardware models and at least two business units. **Why required:** A pilot made of engineers proves only that the package installs for engineers. Real users find the conflicts: the app that clashes with a line-of-business plug-in, the machine with 4 GB of free disk, the profile that lacks a mapped drive. **How to verify:** Entra → **Groups** → `APP – Ring 1 Pilot Devices` exists, has ≥ 10 members, and its membership spans more than one department and more than one hardware model. **How to implement if missing:** Recruit pilot users with their managers' agreement — a named group who understand they receive software first and report problems. Create the group in Implementation Step 2 and populate it before the first release runs through the process.

2.9 A confirmed position on Microsoft Store (new) availability in your cloud

Description: A written, verified statement of whether the **Microsoft Store (new)** app type is available in the Microsoft cloud environment your tenant runs in — Commercial, GCC, GCC High, or DoD — because the format decision matrix branches on the answer. **Why required:** The matrix prefers the Microsoft Store (new) app type where it is available, since it removes packaging effort entirely. **Microsoft Store app availability and feature parity differ between the Commercial cloud and the US Government clouds**, and Government tenants have historically had reduced or absent Store integration alongside delayed feature rollout. If your tenant cannot use the Store type, that branch of the matrix must be struck out and those apps fall through to Win32 — which materially changes your packaging workload. Getting this wrong means a decision tree that recommends an option your admins cannot select. **How to verify:** Sign in to your Intune admin center — `https://intune.microsoft.com` for Commercial, `https://intune.microsoft.us` for GCC High/DoD (portal URLs differ across US Government clouds; **verify the exact URL for your cloud** in Microsoft's US-sovereign-cloud documentation) — then go to **Apps** → **All apps** → **Add** and inspect the **App type** dropdown. Confirm whether **Microsoft Store app (new)** appears and, if it does, whether a search returns results. Cross-check against Microsoft's current service-description and Government feature-parity documentation for your cloud before writing the answer down. **How to implement if missing / if the Store type is unavailable:** Record the finding in your published copy of the standard as *"Microsoft Store (new): NOT AVAILABLE in this cloud — verified <date> by <name>"*, strike the Store branch from your local copy of the format decision matrix, and default those applications to **Win32**. Do not assume a parity gap is permanent; re-verify at each annual review of the standard. **Do not assume the Commercial answer applies to a Government tenant, and do not invent Government specifics — verify for GCC High against Microsoft's documentation.**

2.10 An agreed application catalog taxonomy

Description: The finite list of **Intune app categories** you will allow, agreed before packaging starts, so that Company Portal is browsable and reporting groups sensibly. **Why required:** App categories are free text in practice. Without an agreed list you get `Finance`, `finance`, `Fin Apps`, and `Accounting` as four separate categories within a year, and Company Portal becomes unusable. **How to verify:** Intune admin center → **Apps** → **Categories** (under **Monitor** and setup) → the list matches your agreed taxonomy exactly and contains no near-duplicates. **How to implement if missing:** Agree a short list — this specification ships with a default of six: `Business – Finance`, `Business – Operations`, `Productivity`, `Engineering & Technical`, `Security & Compliance`, `Utilities & Runtimes` — and create them in Implementation Step 2. Fewer is better; you can add a category later, but merging two is manual.

2.11 A record of which existing apps are in scope for retrofit

Description: A baseline inventory of the apps already in your tenant, with their current compliance state against this standard, produced **before** you announce the standard. **Why required:** You cannot mandate a standard without knowing the size of the existing gap. The baseline also becomes the "before" number that proves the standard is working. **How to verify:** Run `.\APPSTD-Invoke-PackageStandardAudit.ps1 -CsvPath C:\Temp\baseline.csv` and confirm the CSV lists every app with a per-rule pass/fail. Keep this file — it is your baseline. **How to implement if missing:** Run the audit script before Implementation Step 10 (enforcement) and file the CSV in `Application Package Records\_Standard\Baseline\`. Decide, per non-compliant app, whether it is **remediate**, **rebuild at next version**, or **accepted exception**.

Checklist

- Intune licensing assigned to all targeted users/devices (2.1)
- Packaging Standard Owner and deputy named (2.2)
- At least two people can package **and** review (2.3)
- Microsoft Graph PowerShell SDK installed on the auditor's workstation (2.4)
- Graph scopes consented for the audit account (2.5)
- `Application Packager (Standard)` role + `SCOPE – Application Packaging` scope tag exist (2.6)
- `Application Package Records` document store created with version history (2.7)
- `APP – Ring 1 Pilot Devices` populated with ≥ 10 real, representative users (2.8)
- Microsoft Store (new) availability **verified in writing** for your cloud (2.9)
- App category taxonomy agreed and created (2.10)
- Baseline audit CSV produced and filed (2.11) — `APPSTD-Invoke-PackageStandardAudit.ps1`

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Application Packaging & Deployment Standards

Follow these steps **in order**. Steps 1–2 create the tenant objects the standard depends on. Steps 3–7 are the specification itself — the rules your packagers follow for every application. Steps 8–10 are the process that makes the rules stick: review, release, and audit.

Unless stated otherwise, every navigation path starts in the **Microsoft Intune admin center** at `https://intune.microsoft.com`. Government-cloud tenants use a different portal host — see Requirement 2.9 and the note at the end of Step 3.

> ***Model:*** *Intake against the spec form → decide the format from the matrix → build to the packaging specification → pass the acceptance checklist → peer review and sign-off → release to Ring 1 pilot → validate → expand to Ring 2 broad → audit quarterly.* ***Assess → Configure → Pilot → Validate → Expand.***

Named objects (reuse these names verbatim everywhere)

Every object below is created in Step 1 or Step 2 and is referenced by name in the later steps, in the Testing Guide, in the Troubleshooting Guide, and by the five PowerShell scripts. Do not improvise names — the scripts match on these patterns.

Object	Exact name to create	Where
Packaging lab ring group	`APP – Ring 0 Packaging Lab`	Entra security group (Assigned, devices)
Pilot ring group	`APP – Ring 1 Pilot Devices`	Entra security group (Assigned, devices)
Broad ring group	`APP – Ring 2 Broad Devices`	Entra security group (Dynamic, devices)
Global exclusion group	`APP – Exclusions – Do Not Deploy`	Entra security group (Assigned, devices)
Approvers group	`ROLE – Package Approvers`	Entra security group (Assigned, users)
Packagers group	`ROLE – Application Packagers`	Entra security group (Assigned, users)
Intune scope tag	`SCOPE – Application Packaging`	Intune scope tag
Intune custom role	`Application Packager (Standard)`	Intune custom RBAC role
App categories (6)	`Business – Finance`, `Business – Operations`, `Productivity`, `Engineering & Technical`, `Security & Compliance`, `Utilities & Runtimes`	Intune app categories
Evidence library	`Application Package Records`	SharePoint document library
Worked-example app	`Acme Software Ltd. Acme Invoice Client – Win32 – v7.3.1`	Intune Win32 app

Step 1 — Create the evidence store

The standard's outputs must land somewhere permanent before you create anything in Intune.

1. Open SharePoint (`https://<yourtenant>.sharepoint.com`) and navigate to your IT team site.
2. Click **+ New** → **Document library**. Name it exactly `Application Package Records`. Click **Create**.
1. Open the library → **Settings** (gear icon) → **Library settings** → **Versioning settings** → set **Create a version each time you edit a file in this document library** to **Yes**. Click **OK**.
1. In the library, click **+ New** → **Folder** and create `_Standard`. Inside `_Standard`, create three folders: `Baseline`, `Exceptions`, and `Audit Reports`.

1. Click **Settings** → **Library settings** → **Permissions for this document library** → **Stop

Inheriting Permissions**. Grant** Contribute **to** `ROLE – Application Packagers`**,** Contribute **to** `ROLE – Package Approvers`**, and** Read** to your audit/compliance group. Remove broad Everyone/Members access.

1. Upload your published copy of this specification to `_Standard\` so the rules and the evidence

live together.

Per-application evidence goes in a folder named `<Vendor> <Product>\<Version>\` — for the worked example, `Acme Software Ltd Acme Invoice Client\7.3.1\`.

Step 2 — Create the tenant objects the standard depends on

2a. The four ring and exclusion groups

1. Go to the Microsoft Entra admin center (`https://entra.microsoft.com`) → **Identity** → **Groups**

→ **All groups** → **+ New group**.

1. **Group type: Security**. **Group name:** `APP – Ring 0 Packaging Lab`. **Group description:**

"Packaging lab devices. First install target for every new package. Never real users." **Membership type: Assigned**. Add your packaging VMs/test devices as members. Click **Create**.

1. Repeat for `APP – Ring 1 Pilot Devices` — description "Representative pilot devices. Every package

installs here and passes the Testing Guide before Ring 2." **Membership type: Assigned.** Add the real pilot devices identified in Requirement 2.8.

1. Repeat for `APP – Ring 2 Broad Devices` — description "The remainder of the managed Windows fleet."

Set **Membership type: Dynamic Device**, then click **Add dynamic query** and enter this rule in the **Rule syntax** box:

```
(device.deviceOSType -eq "Windows") and (device.deviceTrustType -eq "AzureAD")
```

Click **Save**, then **Create**. Adjust the rule to your own trust type if you are hybrid-joined.

1. Repeat for `APP – Exclusions – Do Not Deploy` — description "Devices excluded from all standard app

assignments. Kiosks, lab rigs, and executive-approved exclusions only." **Membership type: Assigned.** Leave it empty for now; it is populated as exclusions are approved.

2b. The two role groups

1. Still in **Entra** → **Groups** → **All groups** → **+ New group**. **Security**, name

`ROLE – Application Packagers`, **Assigned**, members = the staff who build packages. **Create**.

1. Create `ROLE – Package Approvers`, **Security**, **Assigned**, members = the staff permitted to

sign off a release (Requirement 2.3). Open the group → **Owners** → **+ Add owners** → add the Packaging Standard Owner and deputy (Requirement 2.2). **Create**.

2c. The scope tag

1. Intune admin center → **Tenant administration** → **Roles** → **Scope (Tags)** → **+ Create**.

2. **Name:** `SCOPE – Application Packaging`. **Description:** "Objects owned by the application packaging function." Click **Next**.

1. On **Assignments**, click **+ Select groups to include** → select `ROLE – Application Packagers` and `ROLE – Package Approvers`. Click **Next** → **Create**.

2d. The custom packaging role

1. Intune admin center → **Tenant administration** → **Roles** → **All roles** → **+ Create** → **Windows 10 and later** is not what you want here; choose **+ Create** → **Custom role**.

1. **Name:** `Application Packager (Standard)`. **Description:** "Build and edit applications; cannot assign to Ring 2 Broad." Click **Next**.

1. On **Permissions**, expand **Mobile apps** and set **Create**, **Read**, **Update**, and **Delete** to **Yes**. Set **Assign** to **Yes**. Expand **Organization** and set **Read** to **Yes**. Leave everything else **No**. Click **Next**.

1. On **Scope tags**, select `SCOPE – Application Packaging`. Click **Next** → **Create**.

2. Open the new role → **Assignments** → **+ Assign**. **Assignment name:** `Packagers – Standard`. **Members (groups):** `ROLE – Application Packagers`. **Scope (Groups):** select `APP – Ring 0 Packaging Lab` and `APP – Ring 1 Pilot Devices` **only** — this is what prevents a packager from self-releasing to the whole fleet. **Scope (Tags):** `SCOPE – Application Packaging`. Click **Create**.

2e. The app categories

1. Intune admin center → **Apps** → **Categories** → **+ Create**.

2. Enter `Business – Finance` and click **Create**.

3. Repeat for exactly these five more: `Business – Operations`, `Productivity`, `Engineering & Technical`, `Security & Compliance`, `Utilities & Runtimes`.

1. Review the existing list and delete or merge any near-duplicates left over from before the standard (Requirement 2.10). Deleting a category does not delete the apps; you must re-categorise them.

Step 3 — The format decision matrix (THE decision tree)

Rule: every application entering the catalog is assigned exactly one format at intake, by working the decision tree top to bottom and stopping at the first match. The decision and the reason are recorded on the intake form (Step 6). A packager may not change format after sign-off without a new intake.

The decision tree — work it in order

Question 1 — Is the app published in the Microsoft Store and is automatic vendor updating acceptable to the business owner? → **YES:** use **Microsoft Store app (new)**. Stop. → **NO or Store unavailable in your cloud (Requirement 2.9):** continue.

Question 2 — Does the vendor ship a supported MSIX/`.msixbundle`, and does the app need no system-context install, no custom install logic, and no legacy driver/service? → **YES:** use **MSIX**. Stop. → **NO:** continue.

Question 3 — Does the app require a system-context install, custom pre/post logic, a specific install sequence, dependencies, supersedence of a prior version, or a vendor EXE installer? → **YES:** use **Win32**. Stop.

Question 4 — Anything remaining. → Use **Win32**. Win32 is the default; there is no fall-through case that lands on LOB/MSI.

Banned: the **Line-of-business app (`.msi`)** type is **not permitted for new packages**. It offers no control over the install command, no requirement rules, no dependencies, and no supersedence. Existing LOB apps are tracked for migration to Win32 at their next version.

The matrix in full

Criterion	Microsoft Store (new)	MSIX	Win32 (`.intunewin`)	LOB / MSI (legacy)
Packaging effort	None	Low (if vendor ships MSIX)	Moderate — wrap + author rules	Low
Who installs it	Store/OS	OS	Intune Management Extension	MDM client
Custom install command	No	No	**Yes**	No
Custom uninstall command	No	No	**Yes**	No
Detection rules you author	No (managed)	No (managed)	**Yes — required**	No
Requirement rules	Limited	Limited	**Yes**	No
Dependencies	No	No	**Yes**	No
Supersedence	No	No	**Yes**	No
System-context install	Varies	Limited	**Yes**	Yes
Automatic vendor updates	**Yes**	Vendor-dependent	No — you republish	No
Clean uninstall guaranteed	Yes	**Yes** (containerised)	Depends on your command	Usually
Permitted for new packages	**Yes (preferred where available)**	**Yes**	**Yes (default)**	**No**

How to record the decision

On the intake form (Step 6), field **Format decision** must read as a sentence naming the question that decided it, e.g.:

*Format: **Win32**. Decision tree Q3 — vendor ships an EXE bootstrapper, requires system-context install and a VC++ runtime dependency. Not in Microsoft Store; no vendor MSIX available. Decided by J. Okafor, 2026-07-20.*

Government-cloud note

The decision tree's **Question 1** assumes the **Microsoft Store app (new)** type is selectable in your tenant. Microsoft Store integration and broader app-feature parity **differ between the Commercial cloud and the US Government clouds (GCC, GCC High, DoD)**, and Government portal hosts differ as well — Government tenants use hosts such as `intune.microsoft.us` / `portal.azure.us` rather than `intune.microsoft.com` (**verify the exact host for your cloud** against Microsoft's US-sovereign-cloud documentation). If Requirement 2.9 established that the Store type is unavailable in your cloud, **strike Question 1 from your local copy of this tree**; those applications fall through to Question 2 and then to **Win32**, which increases packaging workload and should be reflected in your team's capacity planning. Do not assume the Commercial answer applies — **verify for GCC High** before publishing your local matrix.

Step 4 — The packaging specification (naming, versioning, metadata, documentation)

This is the specification proper. Every package must satisfy all four sub-sections.

4a. Naming convention — applications

Format: `<Publisher> <Product> – <Format> – v<Version>`

- `<Publisher>` — the legal vendor name as the vendor writes it, e.g. `Acme Software Ltd.`
- `<Product>` — the product name without marketing suffixes, e.g. `Acme Invoice Client`
- `<Format>` — exactly one of `Win32`, `MSIX`, `Store`
- `v<Version>` — the vendor's own version string, prefixed `v`, e.g. `v7.3.1`

Worked example: `Acme Software Ltd. Acme Invoice Client – Win32 – v7.3.1`

Rules:

- Use an **en dash surrounded by single spaces** (–) as the separator, consistently. The audit script matches on this.
- **No** dates, ticket numbers, initials, or the words `Test`, `New`, `Copy`, `Final`, `DO NOT USE`, `TEMP`, or `v2 (real)` in a production app name. If a package is a trial, it lives in Ring 0 only and is deleted, not renamed.
- Never reuse an app object for a new version. A new version is a **new app object** with a new version in the name, superseding the old one.

4b. Naming convention — assignment groups and scope tags

Format: `APP – <Purpose> – <Stage>` for rings and app-specific groups; `ROLE – <Function>` for people; `SCOPE – <Function>` for scope tags.

- Ring groups: `APP – Ring 0 Packaging Lab`, `APP – Ring 1 Pilot Devices`, `APP – Ring 2 Broad Devices`
- Exclusions: `APP – Exclusions – Do Not Deploy`
- App-specific targeting group (when an app is not fleet-wide):

`APP – <Product> – Licensed Users` — worked example: `APP – Acme Invoice Client – Licensed Users`

- Never target an app at a group whose name does not begin `APP –` . If you find yourself assigning to `Finance Team (old)`, create a purpose-named `APP –` group instead.

4c. Version stamping

Every package carries its version in **three** places, and all three must agree:

1. The **app name** — … `– v7.3.1` (§4a).
2. The **detection rule** — the rule must assert the version, not merely the presence of the app (Step 5). For the worked example: MSI product code plus a product-version check of `7.3.1`, or a file-version rule of `>= 7.3.1`.
1. The **evidence folder** — `Application Package Records\Acme Software Ltd Acme Invoice Client\7.3.1\`.

A version mismatch between name and detection rule is a **hard fail** of the acceptance checklist. It is the single most common cause of an app that reports Installed while the fleet is on an old build.

4d. Publisher, category, description, and owner metadata

Every app object must have all four populated before release. These are the fields the audit script (`APPSTD-Get-PackageMetadataCompleteness.ps1`) scores.

Field	Rule	Worked example
Publisher	The legal vendor name, identical to the `<Publisher>` token in the app name. Never blank, never "IT", never your own company.	`Acme Software Ltd.`
Category	Exactly one of the six agreed categories (Step 2e). Never blank.	`Business – Finance`
Description	Three mandatory lines, in this order (see below).	*(see below)*
Owner	The business owner's name and department — Intune's **Owner** field on the app. Never a person who has left.	`M. Reyes, Finance`

The **Description** field must contain exactly these three lines, in this order:

```
Business purpose: Submits and approves supplier invoices for the Finance and AP teams.
Business owner: M. Reyes, Finance.
Package record: Application Package Records > Acme Software Ltd Acme Invoice Client > 7.3.1
```

Also set **Developer** to the packager's name and **Information URL** to the vendor's product page. Set **Show this as a featured app in the Company Portal** only for apps the business has asked to promote — it is not a default.

Step 5 — Detection-rule standards

A detection rule is the tenant's only source of truth about whether an app is installed. A weak rule does not "mostly work"; it silently corrupts every install report you will ever run. These rules apply to **Win32** packages (Store and MSIX apps are detected by the platform and have no rule to author).

5a. What makes a detection rule acceptable

A rule is acceptable if it satisfies **all five** tests. The reviewer checks each one at sign-off.

1. **Deterministic** — it returns the same answer on the same machine state every time. No time dependency, no network dependency, no dependency on a user being logged in.
1. **Specific to this app** — it cannot be satisfied by a different application, a different vendor's file, or a shared runtime.
1. **Version-asserting** — it distinguishes v7.3.1 from v7.3.0. Presence-only rules are permitted **only** for apps with no meaningful version (see the exception below) and must be justified on the intake form.
1. **Written by the installer, not by you** — the artefact checked must be something the vendor's installer creates. A registry key your own wrapper script writes is a marker, not detection, and fails if the wrapper succeeds while the install does not.
1. **Verified on a clean machine and on an upgrade** — proven to return *not installed* on a clean Ring 0 VM before install, *installed* after install, and *not installed* when the previous version is present.

5b. The permitted rule types, in order of preference

Preference	Rule type	Use when	Worked example
1 (best)	**MSI product code** + product version	The installer is an MSI	Product code `{A1B2C3D4-…}`, version check **equals** `7.3.1`
2	**File** — path + filename, method **Version**, operator **greater than or equal to**	EXE installers with a versioned binary	`C:\Program Files\Acme\Invoice Client\`, `AcmeInvoice.exe`, version `>= 7.3.1`

Preference	Rule type	Use when	Worked example
3	**Registry** — the vendor's own uninstall key, method **String comparison** on `DisplayVersion`	No reliable versioned file	`HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\{A1B2C3D4-…}`, value `DisplayVersion`, **equals** `7.3.1`
4 (last resort)	**Custom detection script**	Nothing above can express the truth	Must be ≤ 40 lines, exit `0` with STDOUT on detect, exit `0` with no output on not-detected

Exception — presence-only detection. Permitted only when the app genuinely has no version to assert (a certificate, a font pack, a configuration payload). Must be recorded on the intake form as *"Presence-only detection, justified: <reason>"* and countersigned at review.

5c. Banned detection patterns — automatic rejection

A package matching any of these is rejected at review. `APPSTD-Test-DetectionRuleQuality.ps1` flags them.

#	Banned pattern	Why it is banned
B1	**File-exists with no version check** on a versioned app	Reports v7.3.0 as v7.3.1; the fleet never upgrades and the report says everything is fine
B2	**Detecting a folder** rather than a file	Uninstallers routinely leave the folder behind → permanent false "installed"
B3	**Detecting the installer source** (e.g. the `.msi` in a cache path)	Proves you copied a file, not that the app installed
B4	**A registry key your own wrapper writes**	Decouples detection from reality; the wrapper can succeed when the install fails
B5	**A shared runtime or shared DLL** (e.g. `vcruntime140.dll`)	Satisfied by any of a dozen other apps
B6	**Detection on a per-user path (`HKCU`, `%AppData%`) for a system-context install**	Evaluated in SYSTEM context, so it checks the wrong hive/profile and never matches

#	Banned pattern	Why it is banned
B7	**More than one rule type combined without a documented reason**	Ambiguous failure; nobody can tell which clause failed. One authoritative rule per app
B8	**A detection script that writes, installs, repairs, or restarts anything**	Detection must observe, never act. Runs on every evaluation cycle
B9	**A detection script with no explicit exit path** or that relies on `Write-Host`	The IME reads STDOUT and the exit code; `Write-Host` does not reach STDOUT
B10	`Get-CimInstance Win32_Product` in a detection script	Triggers an MSI self-repair consistency check on every call; slow and destructive at scale

5d. Detection rule review evidence

The reviewer records, on the acceptance checklist, the rule type, the exact value, and the three observed results (clean = not detected, post-install = detected, prior-version = not detected).

Step 6 — Install / uninstall command, return-code, and reboot standards

6a. Install and uninstall command standard

Applies to Win32 packages. Every rule is checkable by reading the command line.

1. **Fully silent, no user interaction.** MSI: `/qn`. EXE: the vendor's documented silent switch — found in the vendor's deployment guide, never guessed from a forum post.

1. **Explicitly suppress the installer's own reboot.** MSI: `/norestart`. EXE: the vendor's equivalent. Reboots are decided by the reboot standard (§6c), not by the vendor's installer.

1. **Quote every path containing a space**, and use paths relative to the package root. Never hard-code a path under `C:\Users\` or a mapped drive letter.

1. **Log to a fixed location.** MSI: append `/l*v "%ProgramData%\PackageLogs\<Product>-install.log"`.

Create `C:\ProgramData\PackageLogs\` as part of your build standard. Without a log, triage is guesswork.

1. **Provide a real uninstall command.** MSI: `msiexec /x {PRODUCT-GUID} /qn /norestart`. EXE: the vendor's uninstall string, taken from the uninstall registry key after a test install. An uninstall command of `cmd /c exit 0` is a **hard fail** — it makes Uninstall assignments and supersedence silently lie.

1. **Test both commands on a clean Ring 0 VM** and record the observed exit code on the checklist.

Worked example — `Acme Invoice Client` **7.3.1:**

```
Install:    msiexec /i "AcmeInvoiceClient.msi" /qn /norestart /l*v "%ProgramData%\PackageLogs\AcmeInvoiceC
Uninstall:  msiexec /x {A1B2C3D4-1111-2222-3333-444455556666} /qn /norestart /l*v "%ProgramData%\PackageLo
```

6b. Return-code standard

Every Win32 app must declare its return codes explicitly on the **Program** tab. The defaults are the starting point, not the finished job.

Code	Mapping	Rule
`0`	**Success**	Always present
`1707`	**Success**	Always present (MSI success variant)
`3010`	**Soft reboot**	Always present. Never map to Success — that hides a pending reboot
`1641`	**Hard reboot**	Always present
`1618`	**Retry**	Always present (another install in progress)
Vendor-specific	As documented	Added **only** with a citation to the vendor's documentation, recorded on the intake form

Banned: mapping a non-zero code to **Success** to "make the report green". If an installer returns `1603` and you map it to success, you have not fixed a deployment — you have deleted your only signal. This is a hard fail at review.

6c. Reboot behaviour standard

Set **Device restart behavior** on the **Program** tab according to this table. The default is **No specific action**; anything else needs a reason on the intake form.

Setting	When to use	Notes
No specific action	**Default for all apps.** The `/norestart` switch suppresses the vendor reboot; your update/reboot policy handles restarts.	Use unless there is a documented reason not to
App install may force a device restart	Only when the vendor's installer genuinely cannot be prevented from restarting	Requires a note on the intake form and pilot confirmation
Determine behavior based on return codes	When the vendor documents `3010`/`1641` and you want Intune to honour them	The preferred choice for drivers, agents, and shell extensions
Intune will force a mandatory device restart	**Banned for user-facing apps.**	Permitted only for unattended Ring 0/kiosk devices with the owner's written agreement

Never schedule a Required assignment of a reboot-forcing app to land during business hours.

6d. Install behaviour

System context is the default for line-of-business applications. **User** context is used only when the vendor supports no system install, and it must be noted on the intake form — because per-user installs report per logged-on user and make fleet reporting materially harder.

Step 7 — The Package Intake & Specification Template

This is the reusable form the requester and packager complete **per application, per version**, before any packaging work begins. Copy it into `Application Package Records\<Vendor>\<Product>\<Version>\01-Intake.md`.

```
=========================================================================
PACKAGE INTAKE & SPECIFICATION
=========================================================================
SECTION A — REQUEST (completed by the requester)
  A1  Application name (vendor's own)      : Acme Invoice Client
  A2  Publisher (legal vendor name)        : Acme Software Ltd.
  A3  Version requested                    : 7.3.1
  A4  Business owner (name, department)    : M. Reyes, Finance
  A5  Business purpose (one sentence)      : Submits and approves supplier invoices.
  A6  Who needs it (group / headcount)     : Finance + Accounts Payable, ~85 users
  A7  Licensed? Licence count / evidence   : Yes — 100 seats, PO #44190
  A8  Required by (date) and why           : 2026-09-01, month-end close
  A9  Replaces which existing app?         : Acme Invoice Client v7.2.4
  A10 Data classification handled          : Confidential (supplier bank data)

SECTION B — FORMAT DECISION (completed by the packager)
  B1  In the Microsoft Store? (Q1)         : No
  B2  Vendor MSIX available? (Q2)          : No
  B3  Needs system context / custom logic? : Yes — system context + VC++ runtime
  B4  FORMAT DECIDED                       : Win32
  B5  Decision-tree question that decided  : Q3
  B6  Decided by / date                    : J. Okafor / 2026-07-20

SECTION C — PACKAGE SPECIFICATION (completed by the packager)
  C1  Intune app name (per §4a)            : Acme Software Ltd. Acme Invoice Client – Win32 – v7.3.1
  C2  Category (one of the six)            : Business – Finance
  C3  Source installer + SHA-256           : AcmeInvoiceClient.msi / 9f2c...e10a
  C4  Install command                      : msiexec /i "AcmeInvoiceClient.msi" /qn /norestart /l*v ...
  C5  Uninstall command                    : msiexec /x {A1B2C3D4-...} /qn /norestart /l*v ...
  C6  Install behaviour (System/User)      : System
  C7  Detection rule type + exact value    : MSI product code {A1B2C3D4-...}, version equals 7.3.1
  C8  Detection preference level (1-4)     : 1
  C9  Presence-only? Justification         : No
  C10 Requirement rules                    : 64-bit; minimum Windows 11 22H2
  C11 Dependencies                         : VC++ Redistributable x64 – Win32 – Dependency
  C12 Supersedes                           : Acme Software Ltd. Acme Invoice Client – Win32 – v7.2.4
  C13 Uninstall previous version?          : Yes
  C14 Return codes (non-default + source)  : None beyond standard set
  C15 Reboot behaviour + reason            : No specific action (default)
  C16 Known conflicts / caveats            : Conflicts with Acme Invoice Client 6.x — must be removed

SECTION D — TARGETING
  D1  Ring 0 group                         : APP – Ring 0 Packaging Lab
  D2  Ring 1 group                         : APP – Ring 1 Pilot Devices
  D3  Ring 2 group                         : APP – Acme Invoice Client – Licensed Users
  D4  Exclusions applied                   : APP – Exclusions – Do Not Deploy
  D5  Assignment intent (Required/Avail.)  : Required
=========================================================================
```

A package with an incomplete intake form is not reviewed. That is the point of the form.

Step 8 — The package quality gates (acceptance checklist)

Every package must pass **all** of these before it may be assigned beyond `APP – Ring 0 Packaging Lab`. Any single **FAIL** blocks release. Copy this into `Application Package Records\<Vendor> <Product>\<Version>\02-Acceptance-Checklist.md` and complete it.

Gate 1 — Identity and metadata

#	Check	Pass criteria
1.1	App name matches `<Publisher> <Product> – <Format> – v<Version>`	Exact match, en-dash separators
1.2	No banned words in the name	No `Test`, `New`, `Copy`, `TEMP`, `DO NOT USE`, dates, initials
1.3	Publisher field populated with the legal vendor name	Matches the name token
1.4	Category set to one of the six agreed values	Exact match
1.5	Description contains the three mandatory lines	Business purpose / Business owner / Package record
1.6	Owner and Developer fields populated	Named individuals, currently employed
1.7	Version appears identically in name, detection rule, and evidence folder	All three agree

Gate 2 — Build correctness

#	Check	Pass criteria
2.1	Format matches the intake decision (B4)	Identical
2.2	Source installer hash recorded and matches the file packaged	SHA-256 equal
2.3	Install command fully silent, `/norestart` (or equivalent), logging enabled	Verified on Ring 0
2.4	Uninstall command is real and removes the app	Verified on Ring 0; not a stub
2.5	Install behaviour is System, or User with a documented reason	Matches intake C6

#	Check	Pass criteria
2.6	Return codes declared; no non-zero code mapped to Success	Standard set present
2.7	Reboot behaviour per §6c; forced restart only with owner's agreement	Matches intake C15

Gate 3 — Detection quality

#	Check	Pass criteria
3.1	Rule satisfies all five acceptability tests (§5a)	Reviewer initials each
3.2	Rule matches **no** banned pattern B1–B10 (§5c)	`APPSTD-Test-DetectionRuleQuality.ps1` returns clean
3.3	Exactly one authoritative rule type, unless documented	One rule
3.4	Version asserted, or presence-only justified and countersigned	Evidence recorded
3.5	Three-state proof recorded: clean / post-install / prior-version	All three observed on Ring 0

Gate 4 — Targeting and safety

#	Check	Pass criteria
4.1	Assigned to Ring 0 only at review time	No Ring 1/2 assignment yet
4.2	`APP – Exclusions – Do Not Deploy` is applied as an **Excluded** group	Present on the assignment
4.3	No assignment to **All Devices** or **All Users**	Zero
4.4	Supersedence configured if this replaces a prior version	Matches intake C12/C13
4.5	Dependencies declared with **Automatically install = Yes**	Matches intake C11
4.6	Requirement rules set (architecture, minimum OS)	Matches intake C10

Gate 5 — Evidence

#	Check	Pass criteria
5.1	Intake form complete, all sections	No blanks
5.2	This checklist complete with reviewer name and date	Signed
5.3	Ring 0 install/uninstall test evidence filed	Screenshots or log excerpts
5.4	Source installer and any transforms archived in the evidence folder	Present

Step 9 — Peer review, sign-off, and the release process

9a. The rule

The reviewer must not be the packager. Sign-off is performed by a member of `ROLE – Package Approvers` who did not build the package. If your organisation cannot separate these roles today, that is an entry in the exceptions register with a manager's counter-signature (Requirement 2.3) — not a silent omission.

9b. The review meeting (15 minutes, not a committee)

1. The packager presents the completed intake form and the acceptance checklist.
2. The reviewer opens the app in Intune → **Apps** → **All apps** → the app → and independently

confirms Gates 1, 2, and 4 against what is actually configured, not against what the form claims.

1. The reviewer runs `.\APPSTD-Test-DetectionRuleQuality.ps1 -AppName "<the app name>"` and confirms a

clean result for Gate 3.

1. Any FAIL ends the review. The package returns to the packager; there is no conditional pass.

9c. The sign-off record

On a clean review, the reviewer completes and files `Application Package Records\<Vendor> <Product>\<Version>\03-Signoff.md`:

```
PACKAGE SIGN-OFF
  Application      : Acme Software Ltd. Acme Invoice Client – Win32 – v7.3.1
  Packaged by      : J. Okafor           Date packaged : 2026-07-20
  Reviewed by      : P. Lindqvist        Date reviewed : 2026-07-21
  Gates 1-5        : PASS (all)
  Detection result : APPSTD-Test-DetectionRuleQuality.ps1 — 0 findings
  Exceptions       : None
  Released to      : APP – Ring 1 Pilot Devices
  Ring 2 approval  : Pending pilot validation (Testing Guide)
```

9d. Release to Ring 1

1. Intune admin center → **Apps** → **All apps** → open

`Acme Software Ltd. Acme Invoice Client – Win32 – v7.3.1` → **Properties** → **Assignments** → **Edit**.

1. Under **Required**, click **+ Add group** and select `APP – Ring 1 Pilot Devices`.
2. Under **Excluded**, click **+ Add group** and select `APP – Exclusions – Do Not Deploy`.
3. Click **Review + save** → **Save**.
4. Run the **Testing Guide**. Do not proceed until every gate in it passes.

9e. Release to Ring 2

1. Only after the Testing Guide's pilot gate passes, return to **Assignments** → **Edit**.
2. Under **Required**, click **+ Add group** and select the Ring 2 target from intake field D3 —

for the worked example, `APP – Acme Invoice Client – Licensed Users` (not the whole fleet: this app is licensed for 100 seats).

1. Confirm `APP – Exclusions – Do Not Deploy` is still under **Excluded**. Click **Review + save** → **Save**.

1. Update the sign-off record's **Ring 2 approval** line with the approver's name and date.

9f. Ring and assignment hygiene rules

These are standing rules, checked by `APPSTD-Get-AssignmentHygieneReport.ps1` at every audit.

1. **No app is assigned to All Devices or All Users.** Ever. These are not rings; they are a way to

turn a mistake into an outage.

1. **Every Required assignment carries the** `APP – Exclusions – Do Not Deploy` **exclusion.** One

exclusion group, applied everywhere, so an emergency exclusion is a single membership change.

1. **No app is both Required and Uninstall for overlapping groups.** The result is a device that

installs and uninstalls the app on alternating cycles.

1. **No app is assigned to more than one group with the same intent.** Two Required groups covering

the same devices makes reporting ambiguous; use one purpose-named group.

1. **Every app has at least one assignment or is deleted.** An unassigned app is either an oversight

or debris. Ring 0-only apps older than 30 days are debris.

1. **Ring order is never skipped.** Ring 0 → Ring 1 → Ring 2, with the Testing Guide between 1 and 2.
2. **Superseded apps keep their assignment until supersedence has fully drained**, then are

unassigned and deleted at the next quarterly audit.

1. **App-specific groups are named** `APP – <Product> – <Purpose>` and are owned by a named person.

Step 10 — Enforcement and audit over time

A standard that is not measured is a suggestion. Run this cycle from the day the standard is published.

10a. Baseline (once, before announcing the standard)

```
.\APPSTD-Invoke-PackageStandardAudit.ps1 -CsvPath "C:\Temp\baseline-2026-07.csv"
```

File the CSV in `Application Package Records\_Standard\Baseline\`. This is the number you improve against. Announce the standard **with** the baseline — "we have 62 apps, 41 are non-compliant, here is the plan" lands far better than a policy document with no context.

10b. Per-release enforcement (every package)

The acceptance checklist (Step 8) and peer sign-off (Step 9) are the gate. No package is assigned beyond Ring 0 without a filed sign-off record. The reviewer's signature is the enforcement.

10c. Monthly — new-app conformance

On the first working day of each month, run:

```
.\APPSTD-Invoke-PackageStandardAudit.ps1 -CreatedSinceDays 31 -CsvPath "C:\Temp\newapps.csv"
.\APPSTD-Get-PackageMetadataCompleteness.ps1 -MinimumScore 100
```

Any app created in the last month that fails is chased with its packager that week, while the context is fresh. This is the cheapest possible enforcement and it is where the standard actually holds.

10d. Quarterly — full catalog audit

Run all four read-only scripts and file the outputs in `Application Package Records\_Standard\Audit Reports\<YYYY-Qn>\`:

```
.\APPSTD-Invoke-PackageStandardAudit.ps1        -CsvPath ".\Q3-standard.csv"
.\APPSTD-Test-DetectionRuleQuality.ps1          -CsvPath ".\Q3-detection.csv"
.\APPSTD-Get-AssignmentHygieneReport.ps1        -CsvPath ".\Q3-assignments.csv"
.\APPSTD-Get-PackageMetadataCompleteness.ps1    -CsvPath ".\Q3-metadata.csv"
```

Review the four reports with the Packaging Standard Owner. For each finding, decide: **fix now**, **fix at next version**, or **accepted exception**. Then remediate the mechanical findings:

```
# Preview only — this is the default; nothing is changed
.\APPSTD-Set-AppMetadataStandard.ps1 -CsvPath ".\Q3-metadata.csv"

# Apply, after the preview has been reviewed and approved
.\APPSTD-Set-AppMetadataStandard.ps1 -CsvPath ".\Q3-metadata.csv" -Apply
```

10e. The exceptions register

Not every finding will be fixed, and pretending otherwise is how standards lose credibility. Keep `Application Package Records\_Standard\Exceptions\Exceptions-Register.md` with one row per accepted exception:

App	Rule waived	Reason	Approved by	Approved	Review by
`Legacy Payroll Client – Win32 – v3.0`	§5a.3 version-asserting detection	Vendor writes no version anywhere; presence-only is the only option	Packaging Standard Owner	2026-07-21	2027-01-21

Every exception has an expiry date. An exception with no review date is a permanent hole.

10f. Annual review of the standard itself

Once a year the Packaging Standard Owner reviews this specification against reality: re-verify the Microsoft Store availability position (Requirement 2.9), re-check the format decision tree against current Microsoft capabilities, confirm the category taxonomy is still right, and close or renew every exception. Record the review date on the front page of your published copy.

Order of operations

`Evidence store → Tenant objects (groups, scope tag, role, categories) → Format decision matrix → Packaging specification (naming, versioning, metadata) → Detection standards → Command / return-code / reboot standards → Intake template → Acceptance checklist → Peer review + sign-off → Ring 1 → validate → Ring 2 → monthly + quarterly audit → annual standard review`

5. Testing Guide — Application Packaging & Deployment Standards

This guide validates two things: (a) that the **standard's tenant objects and controls exist and work**, and (b) that **an individual package correctly passes through the standard** end to end. Run Part A once, when you adopt the standard. Run Part B for every package, using the worked example `Acme Software Ltd. Acme Invoice Client – Win32 – v7.3.1` as the model.

Part A — The standard is in place

Test A1 — All named objects exist

Do: In Entra → **Groups**, confirm `APP – Ring 0 Packaging Lab`, `APP – Ring 1 Pilot Devices`, `APP – Ring 2 Broad Devices`, `APP – Exclusions – Do Not Deploy`, `ROLE – Package Approvers`, and `ROLE – Application Packagers` all exist. In Intune → **Tenant administration** → **Roles** → **Scope (Tags)** confirm `SCOPE – Application Packaging`; under **All roles** confirm `Application Packager (Standard)`. In **Apps** → **Categories** confirm the six categories. **Expected:** Every object is present with the exact name. Any missing object is a Step 2 gap.

Test A2 — The packager role cannot reach Ring 2

Do: Sign in as a member of `ROLE – Application Packagers` (or review the role assignment). Confirm the role's assignment scope lists **only** `APP – Ring 0 Packaging Lab` and `APP – Ring 1 Pilot Devices`. **Expected:** A packager can assign to Ring 0 and Ring 1 but **not** Ring 2 — the release gate is enforced by permissions, not just process.

Test A3 — The audit scripts connect and return data

Do: Run `.\APPSTD-Invoke-PackageStandardAudit.ps1 -WhatIfOnlySummary`. **Expected:** The script connects to Graph, enumerates apps, and prints a compliant/non-compliant summary with no error. This confirms Requirements 2.4 and 2.5.

Test A4 — The evidence store is ready

Do: Open the `Application Package Records` SharePoint library. Confirm version history is on, `_Standard\Baseline`, `_Standard\Exceptions`, and `_Standard\Audit Reports` exist, and

permissions grant Contribute to packagers/approvers and Read to audit. **Expected:** The store exists and is access-controlled (Requirement 2.7).

Part B — A package passes through the standard

Test B1 — Intake form is complete before packaging

Do: Open `Application Package Records\Acme Software Ltd Acme Invoice Client\7.3.1\01-Intake.md`. **Expected:** Sections A–D are complete, the format decision (B4) names the deciding question, and no field is blank. An incomplete form means the package should not have been built yet.

Test B2 — Naming, version stamping, and metadata are compliant

Do: Run `.\APPSTD-Invoke-PackageStandardAudit.ps1 -CreatedSinceDays 7` and find the app, then run `.\APPSTD-Get-PackageMetadataCompleteness.ps1 -MinimumScore 100`. **Expected:** The app reports `Compliant: YES` with zero violations, and a metadata score of `100`. The version `7.3.1` appears identically in the name, the detection rule, and the evidence folder.

Test B3 — Detection rule passes the quality checker

Do: Run `.\APPSTD-Test-DetectionRuleQuality.ps1 -AppName "Acme Software Ltd. Acme Invoice Client – Win32 – v7.3.1"`. **Expected:** `Result: PASS`, `FindingCount: 0`. No banned pattern (B1–B10) is triggered. This is the automated half of acceptance Gate 3.

Test B4 — Three-state detection proof on Ring 0

Do: On a clean `APP – Ring 0 Packaging Lab` VM: (1) confirm detection reports **not installed** before install; (2) install and confirm detection reports **installed**; (3) on a VM with v7.2.4 present, confirm the v7.3.1 rule reports **not installed** (i.e. it distinguishes versions). **Expected:** All three states observed and recorded on the acceptance checklist (Gate 3.5).

Test B5 — Commands are silent and reversible

Do: On a Ring 0 VM, run the install command manually, then the uninstall command. Check `C:\ProgramData\PackageLogs\` for the logs. **Expected:** Both run with no UI, return exit `0`, and the uninstall genuinely removes the app (detection then reports not installed). A stub uninstall fails this test (Gate 2.4).

Test B6 — Assignment hygiene is clean

Do: After the app is assigned to Ring 1, run `.\APPSTD-Get-AssignmentHygieneReport.ps1`. **Expected:** The app reports `Clean: YES`. The `APP – Exclusions – Do Not Deploy` group is present as an **Excluded** group, and there is no All Devices/All Users target and no Required+Uninstall conflict.

Test B7 — Sign-off record exists before Ring 2

Do: Open `03-Signoff.md` in the evidence folder. **Expected:** It names a reviewer who is **not** the packager, records Gates 1–5 as PASS, cites the detection checker's 0-finding result, and shows Ring 2 approval as pending until pilot validation.

Test B8 — Pilot install report

Do: After Ring 1 assignment and a policy cycle, in Intune open the app → **Device install status**.
Expected: Pilot devices report **Installed** (not "Installed but failed", not a reinstall loop), confirming the detection rule reports the truth on real devices.

Pass/fail gate

Gate	Requirement
Standard-ready	Tests A1–A4 pass — objects exist, packager role is scoped, scripts run, evidence store ready
Pilot pass (per package)	Tests B1–B8 pass on `APP – Ring 1 Pilot Devices`; detection checker 0 findings; signed sign-off filed
Broad pass (per package)	Test B8 reports Installed across `APP – Ring 2 Broad Devices` after expansion; hygiene report clean

Do **not** assign a package to Ring 2 until its pilot gate passes and the sign-off record is filed.

6. Troubleshooting Guide — Application Packaging & Deployment Standards

Use `APPSTD-Invoke-PackageStandardAudit.ps1` for naming/metadata violations, `APPSTD-Test-DetectionRuleQuality.ps1` for detection findings, `APPSTD-Get-AssignmentHygieneReport.ps1` for assignment problems, `APPSTD-Get-PackageMetadataCompleteness.ps1` for metadata scores, and `APPSTD-Set-AppMetadataStandard.ps1` (preview by default) to normalise mechanical gaps. The worked example throughout is `Acme Software Ltd. Acme Invoice Client – Win32 – v7.3.1`.

A. The scripts won't connect or return nothing

Symptom	Cause	Remediation
`Connect-MgGraph` fails or prompts endlessly	SDK missing or scopes not consented	Install the SDK (Requirement 2.4); run once interactively as an Intune Administrator to accept the consent prompt (Requirement 2.5)
Script runs but reports "No matching Windows apps found"	Tenant genuinely has no Win32/Store/MSIX apps, or a filter excluded them	Confirm apps exist in Intune → **Apps** → **All apps**; remove `-CreatedSinceDays` if you set it
`Insufficient privileges` on the remediation `-Apply`	Only read scope consented	Re-run and consent to `DeviceManagementApps.ReadWrite.All`; the account needs the **Intune Administrator** role

Symptom	Cause	Remediation
Consent blocked by the tenant	User-consent is disabled	Have a Global Administrator grant admin consent for the Microsoft Graph PowerShell app for the listed scopes

B. An app fails the naming/standard audit but "looks fine"

Symptom	Cause	Remediation
`name-convention` FAIL on a correct-looking name	The separator is a plain hyphen without surrounding spaces, or the format token isn't exactly `Win32`/`MSIX`/`Store`	Rename to `<Publisher> <Product> – <Format> – v<Version>` with – (space-dash-space); a new version is a new app object, never an in-place rename of a live one
`version-stamp` FAIL	No `vX.Y` token in the name	Add the vendor version, prefixed `v`, to the name and confirm the detection rule asserts the same version
`banned-word` FAIL for `New`/`Copy`	A real word in the product name coincides with a banned token	This is a false positive on a legitimate name; record it as an accepted exception in the register rather than renaming the product
`description-standard` FAIL	Description lacks the three mandatory lines	Use `APPSTD-Set-AppMetadataStandard.ps1` with `BusinessPurpose`/`BusinessOwner`/`PackageRecord` values (preview, then `-Apply`)

C. Detection quality checker flags a rule

Finding	Meaning	Remediation
`B1(file-exists-no-version)`	A versionable app is detected by mere existence	Change the file rule's method to **Version** with operator ≥, or use the MSI product code with a version check
`B2(folder-only)`	Detection points at a folder, not a file	Point at a specific versioned file the installer writes; uninstallers leave folders behind
`B5(shared-runtime)`	Detection targets a shared DLL/runtime	Re-target the app's own uniquely-named binary or its MSI product code
`B6(per-user-path-system-context)`	An `HKCU`/`%AppData%` rule on a System-context install	Move detection to an `HKLM` key or a `Program Files` file; SYSTEM cannot see the user hive

Finding	Meaning	Remediation
`B7(multiple-rule-types)`	More than one rule type combined	Reduce to one authoritative rule, or document the reason and countersign
`B8`/`B9`/`B10` on a script rule	The detection script acts, uses `Write-Host`/no exit, or calls `Win32_Product`	Rewrite the script to only observe: emit STDOUT + exit `0` on detect, no output + exit `0` on not-detected; never `Win32_Product`
`NO-RULE(missing detection)`	The Win32 app has no detection rule	Add one before any assignment; without detection Intune cannot report state

D. Assignment hygiene report shows violations

Finding	Meaning	Remediation
`H1(all-devices/all-users)`	App targets a virtual all-fleet group	Remove it; assign to `APP – Ring 1/2` or a purpose-named `APP –` group instead
`H2(required+uninstall)`	Same app is both Required and Uninstall for overlapping groups	The device installs and uninstalls on alternating cycles; remove one intent and let supersedence handle replacement
`H3(duplicate-intent-target)`	Two Required (or two Available) assignments to the same group	Delete the duplicate; one assignment per intent per group
`H4(missing-standard-exclusion)`	A Required assignment lacks `APP – Exclusions – Do Not Deploy`	Add the exclusion group under **Excluded** on the assignment
`H5(unassigned-orphan)`	App has no assignments	Assign it, or delete it — a Ring 0-only app older than 30 days is debris

E. Metadata remediation didn't change anything

• **By design, it previews by default.** Nothing is written until you pass `-Apply`. Run once to review the diff, then re-run with `-Apply`.

• **A category name "not found in tenant"** means the exact category string isn't in **Apps → Categories**. Create it first (Implementation Step 2e); the script matches on the exact display name.

• **The description overwrote free text.** The script preserves only text placed **below** the

`<!-- free-text below this line is preserved -->` marker. Put any narrative below that marker.

F. The standard is "adopted" but nobody follows it

This is the most common failure, and it is not technical.

- **Symptom:** new apps still arrive non-compliant. **Cause:** the acceptance checklist and sign-off

are being skipped because there is no consequence. **Remediation:** enforce the packager RBAC scope (Requirement 2.6) so packagers physically cannot self-release to Ring 2, and run the monthly new-app audit (Implementation Step 10c) with the Packaging Standard Owner reviewing the results.

- **Symptom:** the exceptions register is full of permanent, never-reviewed waivers. **Cause:**

exceptions were logged without expiry dates. **Remediation:** add a review date to every exception; close or renew each at the annual review (Step 10f).

- **Symptom:** the baseline never improves quarter over quarter. **Cause:** findings are reported but

not assigned to anyone. **Remediation:** each quarterly finding gets a decision — fix now, fix at next version, or accepted exception — with an owner and a date.

G. Government-cloud specifics

- **Portal host differs.** Government tenants use hosts such as `intune.microsoft.us` / `portal.azure.us`

rather than `.com`. Confirm the exact host for your cloud in Microsoft's US-sovereign-cloud documentation before following any portal path.

- **The Microsoft Store (new) branch may be unavailable.** If Requirement 2.9 established the Store

type is absent in your cloud, an app the matrix would route to Store must fall through to Win32. Strike Question 1 from your local decision tree and reflect the extra Win32 workload in capacity planning. **Verify for GCC High** against Microsoft's documentation; do not assume the Commercial answer, and do not invent Government specifics.

H. When to escalate

If the naming, detection, metadata, and assignment audits all report clean, the acceptance checklist passed, sign-off is filed, the pilot reports Installed — but Intune still reports Failed for an app across the fleet after a policy cycle — the problem is in the per-app package mechanics (install command, dependency, or the installer itself), not in the standard. Collect the device's `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log` and the installer's own verbose log (`C:\ProgramData\PackageLogs\`) and treat it as a per-app packaging fault (see the companion Win32 packaging solution) or open a Microsoft support case.

7. Best Practices — Application Packaging & Deployment Standards

• **Name the app once, correctly, forever.** `<Publisher> <Product> – <Format> – v<Version>` with

en-dash separators, no ticket numbers, no `Test`/`Copy`/`Final`. A catalog you can search is a catalog you can audit; a catalog of one-off names is a liability that grows with every package.

• **The version must agree in three places.** The app name, the detection rule, and the evidence

folder all carry `v7.3.1`. A version that appears in the name but not the detection rule is the number-one cause of a fleet that reports Installed while running last quarter's build.

• **Decide the format at intake, in writing, from the tree.** Work the decision tree top to bottom

and record the deciding question on the intake form. Prefer Microsoft Store (new) where it is available and auto-update is acceptable, then MSIX, then Win32 as the default. **Never** create a new LOB/MSI app — it is banned for new packages.

• **Detection observes, never acts.** A detection rule or script must be deterministic, specific,

version-asserting, written by the vendor's installer (not your wrapper), and proven on a clean machine. Never `Win32_Product` (it triggers MSI self-repair on every cycle), never a folder-only check, never a shared runtime, never `HKCU` for a system-context install.

• **One authoritative detection rule per app.** Combining rule types produces ambiguous failures

nobody can diagnose. If you genuinely need more than one, document why on the intake form and have it countersigned.

• **Never map a non-zero exit code to Success.** `1603` mapped to Success does not make a deployment

work; it deletes your only signal that it failed. Declare `0`, `1707`, `3010`, `1641`, `1618` explicitly and add vendor codes only with a citation.

• **Let return codes and policy own reboots.** Use `/norestart`, set **No specific action** by

default, and reserve forced restarts for unattended Ring 0/kiosk devices with the owner's written agreement. Never schedule a reboot-forcing Required app to land during business hours.

• **The reviewer is never the packager.** Sign-off is a second person from `ROLE – Package Approvers`

who independently confirms the gates against what Intune actually shows, not against what the form claims. If you cannot separate the roles, record it as an exception with a manager's counter-signature — do not quietly skip the gate.

• **One exclusion group, applied to every Required assignment.** `APP – Exclusions – Do Not Deploy`

on every deployment means an emergency exclusion is a single membership change, not an edit to fifty apps.

• **Never assign to All Devices or All Users.** They are not rings; they are the shortest path from a

mistake to a fleet-wide outage. Assign to `APP –` purpose-named groups only, in ring order.

• **Retain the evidence, not just the package.** The intake form, the acceptance checklist, the

sign-off record, and the three-state detection proof live in `Application Package Records` for the app's lifetime. "Who approved this, when, against what test?" must have a filed answer.

- **Audit on a cadence, remediate with a preview.** Baseline before you announce, check new apps monthly, audit the whole catalog quarterly, and re-verify the standard itself annually. Run the metadata remediation script in its default preview mode first and only `-Apply` after the diff is reviewed.
- **Keep an honest exceptions register with expiry dates.** Not everything gets fixed; an exception with no review date is a permanent hole pretending to be a decision. Every waiver names an owner, a reason, and a date it will be revisited.
- **Re-verify Government-cloud format availability.** The Microsoft Store (new) app type and some app features have parity gaps in US Government clouds and different portal hosts. Confirm availability for your cloud before you commit the Store branch of the matrix, and re-check it at each annual review. Verify for GCC High against Microsoft's documentation — never assume the Commercial answer.

8. Deliverables Summary — Application Packaging & Deployment Standards

Documentation

Document	Purpose
Executive Summary	Business case: why an app catalog decays without a standard, and what this fixes
1. Solution Overview	The standards layer above per-app packaging; four control points; the three app formats; what the standard produces
2. Requirements Guide	Every prerequisite (owner, two reviewers, Graph SDK/scopes, RBAC, evidence store, pilot population, Government Store position, taxonomy, baseline) with Verify / Implement-if-missing
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Create the tenant objects; the format decision matrix + tree; the packaging specification (naming, versioning, metadata); detection-rule standards + banned patterns; command/return-code/reboot standards; the intake template; the acceptance checklist; peer review & sign-off; ring/assignment hygiene; enforcement & audit
5. Testing Guide	Validates the standard is in place (Part A) and that a package passes through it (Part B), with pass/fail gates
6. Troubleshooting Guide	Field issues with the standard, the scripts, and packages that fail their gates

Document	Purpose
7. Best Practices	Field-proven rules that keep the catalog clean at scale

PowerShell scripts (Microsoft Graph, read-only by default)

Script	What it does
`APPSTD-Invoke-PackageStandardAudit.ps1`	Scans every app for naming-convention, banned-word, publisher, version-stamp, description-standard, and legacy-LOB violations; console summary + optional CSV baseline (read-only)
`APPSTD-Test-DetectionRuleQuality.ps1`	Reads Win32 detection rules and flags the banned patterns B1–B10 (folder-only, no-version, shared runtime, per-user path in system context, multi-rule, acting/`Win32_Product`/`Write-Host` scripts); the automated half of acceptance Gate 3 (read-only)
`APPSTD-Get-AssignmentHygieneReport.ps1`	Reports assignment violations: All Devices/All Users targeting, Required+Uninstall conflicts, duplicate same-intent assignments, missing standard exclusion group, and orphan apps (read-only)
`APPSTD-Get-PackageMetadataCompleteness.ps1`	Scores each app 0–100 on publisher, category, the three-line standardised description, developer/owner, info URL, and logo; lists apps below threshold (read-only)
`APPSTD-Set-AppMetadataStandard.ps1`	Normalises publisher, category, and the standardised description block from a CSV; **preview by default**, supports `-WhatIf`, changes nothing until `-Apply`; never touches detection, commands, or assignments

Reusable templates (inside the Implementation Guide)

- **Package Intake & Specification Template** (Step 7) — the per-app, per-version form completed before packaging.
- **Package Acceptance Checklist** (Step 8) — the five-gate quality gate every package must pass.
- **Package Sign-off Record** (Step 9) — the retained evidence of peer review and release approval.
- **Exceptions Register** (Step 10e) — the honest log of accepted waivers with expiry dates.

What the buyer receives

- This specification as a **Word** document and a **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable standard following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

An organisation-wide packaging standard that keeps the app catalog consistent, auditable, and clean as it grows — every app correctly named and version-stamped, every detection rule provably correct, every release peer-reviewed and evidenced, every assignment hygienic, and the whole catalog audited on a cadence by the included scripts. This is the specification layer **above** per-app packaging; it governs how every application is delivered, not how any single one is wrapped.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

APPSTD-Get-AssignmentHygieneReport.ps1

Purpose: Read-only. Enumerates every managed Windows app and its assignments and applies the standard's ring/assignment hygiene rules (Implementation Step 9f): - H1 Assigned to All Devices or All Users (virtual groups) -> violation. - H2 Required AND Uninstall intent for overlapping targets -> conflict. - H3 Two or more assignments with the same intent (duplicate) -> ambiguity. - H4 Required assignment missing the 'APP – Exclusions – Do Not Deploy' exclusion. - H5 No assignments at all (orphan) — flagged, with age if available. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPSTD-Get-AssignmentHygieneReport.ps1 powershell -ExecutionPolicy Bypass -File .\APPSTD-Get-AssignmentHygieneReport.ps1 -CsvPath C:\Temp\assignments.csv powershell -ExecutionPolicy Bypass -File .\APPSTD-Get-AssignmentHygieneReport.ps1 -ExclusionGroupName "APP – Exclusions – Do Not Deploy" Requires the Microsoft Graph PowerShell SDK, DeviceManagementApps.Read.All and Group.Read.All. EXPECTED RESULTS A per-app hygiene report and a summary count per rule. Nothing is changed — read-only. .NOTES Intune Solutions — read-only against Microsoft Graph. No changes are made to the tenant. > [CmdletBinding()] param([string]$CsvPath, [string]$ExclusionGroupName = 'APP – Exclusions – Do Not Deploy')

How to run: powershell -ExecutionPolicy Bypass -File .\APPSTD-Get-AssignmentHygieneReport.ps1 powershell -ExecutionPolicy Bypass -File .\APPSTD-Get-AssignmentHygieneReport.ps1 -CsvPath C:\Temp\assignments.csv powershell -ExecutionPolicy Bypass -File .\APPSTD-Get-AssignmentHygieneReport.ps1 -ExclusionGroupName "APP – Exclusions – Do Not Deploy" Requires the Microsoft Graph PowerShell SDK, DeviceManagementApps.Read.All and Group.Read.All. EXPECTED RESULTS A per-app hygiene report and a summary count per rule. Nothing is changed — read-only. .NOTES Intune Solutions — read-only against Microsoft Graph. No changes are made to the tenant. > [CmdletBinding()] param([string]$CsvPath, [string]$ExclusionGroupName = 'APP – Exclusions – Do Not Deploy')

APPSTD-Get-PackageMetadataCompleteness.ps1

Purpose: Read-only. For every managed Windows app it scores the metadata fields the standard requires before release (Implementation Step 4d): - Publisher present and not a placeholder (weight 20) - App category assigned (weight 20) - Description has the three mandatory lines (weight 25) Business purpose: / Business owner: / Package record: - Developer (packager) set (weight 10) - Owner (business owner) set (weight 10) - Information URL set (weight 10) - Logo/icon present (weight 5) Apps scoring below -MinimumScore are listed for remediation. Feed the CSV to APPSTD-Set-AppMetadataStandard.ps1 to normalise the mechanical gaps. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPSTD-Get-PackageMetadataCompleteness.ps1 powershell -ExecutionPolicy Bypass -File .\APPSTD-Get-PackageMetadataCompleteness.ps1 -MinimumScore 100 -CsvPath C:\Temp\metadata.csv Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A per-app score table and a list of apps below the threshold, plus a CSV for remediation. Nothing is changed — read-only. .NOTES Intune Solutions — read-only against Microsoft Graph. No changes are made to the tenant. > [CmdletBinding()] param([ValidateRange(0,100)][int]$MinimumScore = 100, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\APPSTD-Get-PackageMetadataCompleteness.ps1 powershell -ExecutionPolicy Bypass -File .\APPSTD-Get-PackageMetadataCompleteness.ps1 -MinimumScore 100 -CsvPath C:\Temp\metadata.csv Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A per-app score table and a list of apps below the threshold, plus a CSV for remediation. Nothing is changed — read-only. .NOTES Intune Solutions — read-only against Microsoft Graph. No changes are made to the tenant. > [CmdletBinding()] param([ValidateRange(0,100)][int]$MinimumScore = 100, [string]$CsvPath)

APPSTD-Invoke-PackageStandardAudit.ps1

Purpose: Read-only. Enumerates all managed apps via Microsoft Graph and scores each against the Application Packaging & Deployment Standard: - Name matches "<Publisher> <Product> - <Format> - v<Version>" (en-dash separated). - Name is free of banned words (Test, New, Copy, TEMP, DO NOT USE, Final, dates, initials). - Publisher populated and not a placeholder (IT, Unknown, blank). - Category (app category) assigned. - Version present in the app name for versionable formats. Produces a per-app pass/fail summary and, optionally, a CSV baseline. Use it once to baseline before announcing the standard, monthly for new apps, and quarterly for the catalog. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPSTD-Invoke-PackageStandardAudit.ps1 powershell -ExecutionPolicy Bypass -File .\APPSTD-Invoke-PackageStandardAudit.ps1 -CsvPath C:\Temp\baseline.csv powershell -ExecutionPolicy Bypass -File .\APPSTD-Invoke-PackageStandardAudit.ps1 -CreatedSinceDays 31 Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A console summary (compliant vs non-compliant counts, top violations) and, with -CsvPath, a per-app CSV with a column per rule. Nothing is changed — this script only reads. .NOTES Intune Solutions — read-only against Microsoft Graph. No changes are made to the tenant. > [CmdletBinding()] param([string]$CsvPath, [int]$CreatedSinceDays, [switch]$WhatIfOnlySummary)

How to run: powershell -ExecutionPolicy Bypass -File .\APPSTD-Invoke-PackageStandardAudit.ps1 powershell -ExecutionPolicy Bypass -File .\APPSTD-Invoke-PackageStandardAudit.ps1 -CsvPath C:\Temp\baseline.csv powershell -ExecutionPolicy Bypass -File .\APPSTD-Invoke-PackageStandardAudit.ps1 -CreatedSinceDays 31 Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A console summary (compliant vs non-compliant counts, top violations) and, with -CsvPath, a per-app CSV with a column per rule. Nothing is changed — this script only reads. .NOTES Intune Solutions — read-only against Microsoft Graph. No changes are made to the tenant. > [CmdletBinding()] param([string]$CsvPath, [int]$CreatedSinceDays, [switch]$WhatIfOnlySummary)

APPSTD-Set-AppMetadataStandard.ps1

Purpose: The only write-capable script in this package, and it changes nothing until you explicitly pass -Apply. It supports -WhatIf. It normalises the mechanical, low-risk metadata gaps the audit finds: - Trims/repairs the Publisher field when a mapping is supplied. - Assigns an app category when supplied. - Rebuilds the three-line standardised Description (Business purpose / Business owner / Package record) from supplied values, PRESERVING any existing free-text below a marker. It does NOT touch detection rules, commands, assignments, or supersedence — those are deliberate, reviewed changes, never bulk-normalised. Input is a CSV you author (or the metadata report annotated with target values). Each row: AppName,Publisher,Category,BusinessPurpose,BusinessOwner,PackageRecord Blank cells are skipped (that field is left unchanged). HOW TO RUN 1) Preview (default — safe, no changes): powershell -ExecutionPolicy Bypass -File .\APPSTD-Set-AppMetadataStandard.ps1 -CsvPath .\fixes.csv 2) Preview a

single app inline: powershell -ExecutionPolicy Bypass -File .\APPSTD-Set-AppMetadataStandard.ps1 -AppName "Acme ..." -Publisher "Acme Software Ltd." -Category "Business – Finance" 3) Apply, after the preview has been reviewed: powershell -ExecutionPolicy Bypass -File .\APPSTD-Set-AppMetadataStandard.ps1 -CsvPath .\fixes.csv -Apply Requires the Microsoft Graph PowerShell SDK, DeviceManagementApps.ReadWrite.All (for -Apply). EXPECTED RESULTS Default: a table of the exact changes that WOULD be made (old -> new), nothing altered. With -Apply: the same table, then each change PATCHed to Graph with a per-app success line. .NOTES Intune Solutions — write-capable but PREVIEW BY DEFAULT (-WhatIf supported; -Apply required to change anything). Only metadata is touched; detection, commands, and assignments are never changed. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$CsvPath, [string]$AppName, [string]$Publisher, [string]$Category, [string]$BusinessPurpose, [string]$BusinessOwner, [string]$PackageRecord, [switch]$Apply)

How to run: 1) Preview (default — safe, no changes): powershell -ExecutionPolicy Bypass -File .\APPSTD-Set-AppMetadataStandard.ps1 -CsvPath .\fixes.csv 2) Preview a single app inline: powershell -ExecutionPolicy Bypass -File .\APPSTD-Set-AppMetadataStandard.ps1 -AppName "Acme ..." -Publisher "Acme Software Ltd." -Category "Business – Finance" 3) Apply, after the preview has been reviewed: powershell -ExecutionPolicy Bypass -File .\APPSTD-Set-AppMetadataStandard.ps1 -CsvPath .\fixes.csv -Apply Requires the Microsoft Graph PowerShell SDK, DeviceManagementApps.ReadWrite.All (for -Apply). EXPECTED RESULTS Default: a table of the exact changes that WOULD be made (old -> new), nothing altered. With -Apply: the same table, then each change PATCHed to Graph with a per-app success line. .NOTES Intune Solutions — write-capable but PREVIEW BY DEFAULT (-WhatIf supported; -Apply required to change anything). Only metadata is touched; detection, commands, and assignments are never changed. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$CsvPath, [string]$AppName, [string]$Publisher, [string]$Category, [string]$BusinessPurpose, [string]$BusinessOwner, [string]$PackageRecord, [switch]$Apply)

APPSTD-Test-DetectionRuleQuality.ps1

Purpose: Read-only. For each Win32 app (or one named app) it reads the detectionRules and evaluates: - B1 File-exists rule with no version/size check on a versionable app. - B2 Detection targets a folder (no file name). - B4/B5 Registry/file detection on a shared runtime or generic marker path. - B6 Per-user path (HKCU / %AppData% / %LocalAppData%) for a system-context install. - B7 More than one detection rule type combined. - B8/B9/B10 Detection SCRIPT that appears to write/act, lacks an exit path, uses Write-Host, or calls Win32_Product (MSI self-repair). Also confirms the acceptability signal that the rule asserts a version where one is expected. This is the automated half of Acceptance-Checklist Gate 3; the reviewer still initials §5a. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPSTD-Test-DetectionRuleQuality.ps1 powershell -ExecutionPolicy Bypass -File .\APPSTD-Test-DetectionRuleQuality.ps1 -AppName "Acme Software Ltd. Acme Invoice Client – Win32 – v7.3.1" powershell -ExecutionPolicy Bypass -File .\APPSTD-Test-DetectionRuleQuality.ps1 -CsvPath C:\Temp\detection.csv Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS Per app: PASS with 0 findings, or a list of the banned-pattern codes triggered. A clean result (0 findings) is what Gate 3.2 requires before sign-off. .NOTES Intune Solutions — read-only against Microsoft Graph. No changes are made to the tenant. > [CmdletBinding()] param([string]$AppName, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\APPSTD-Test-DetectionRuleQuality.ps1 powershell -ExecutionPolicy Bypass -File .\APPSTD-Test-DetectionRuleQuality.ps1 -AppName "Acme Software Ltd. Acme Invoice Client – Win32 – v7.3.1" powershell -ExecutionPolicy Bypass -File

.\APPSTD-Test-DetectionRuleQuality.ps1 -CsvPath C:\Temp\detection.csv Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS Per app: PASS with 0 findings, or a list of the banned-pattern codes triggered. A clean result (0 findings) is what Gate 3.2 requires before sign-off. .NOTES Intune Solutions — read-only against Microsoft Graph. No changes are made to the tenant. > [CmdletBinding()] param([string]$AppName, [string]$CsvPath)